Sabancı Üniversitesi

Multimodal Human–AI Interfaces for Mobile AR

Software Infrastructure & Real-Time Pipeline Analysis

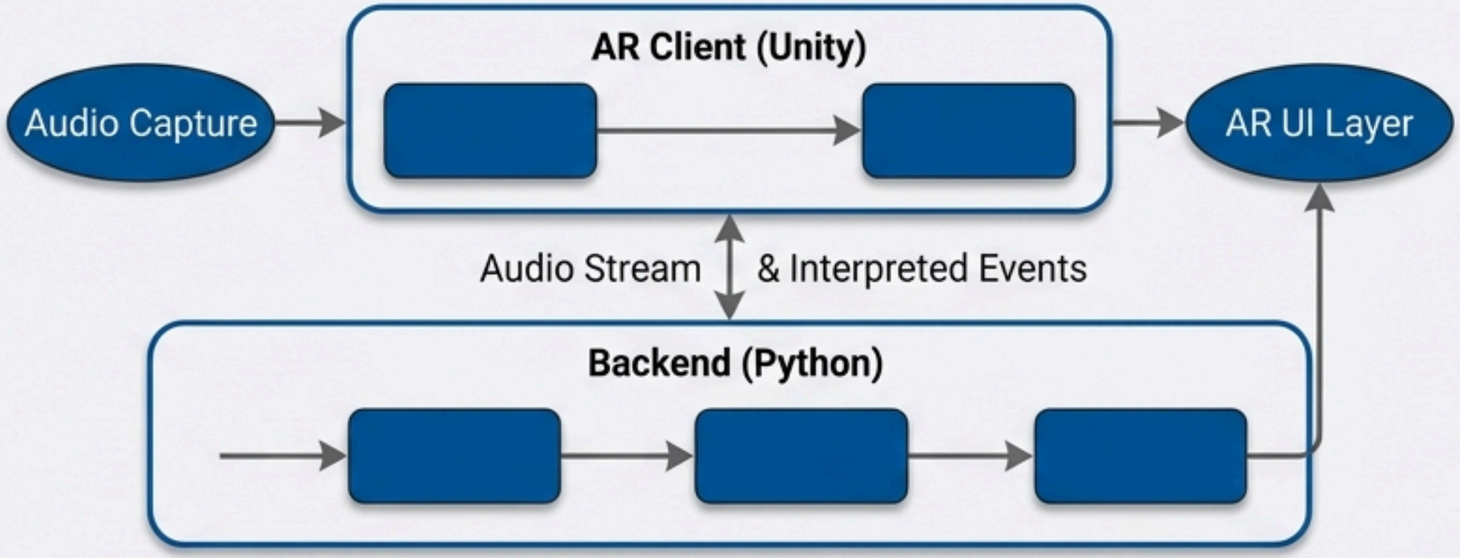
Alper Çamlı
Mohamad Samer Bader
Aynur Aybüke Memiş

Supervisor: Selim Balcısoy

System Goals and Mobile Application Scope

The Problem:
DHH individuals miss critical environmental sounds (alarms, traffic) and lack context in dynamic environments.

The Solution:
A handheld Mobile AR Application (Android) that visualizes sound in real-time.



Core Functions

1. Detect: Identify safety-critical sounds (e.g., sirens)

2. Transcribe: Speech-to-Text for conversation

3. Contextualize: AI summaries of the environment

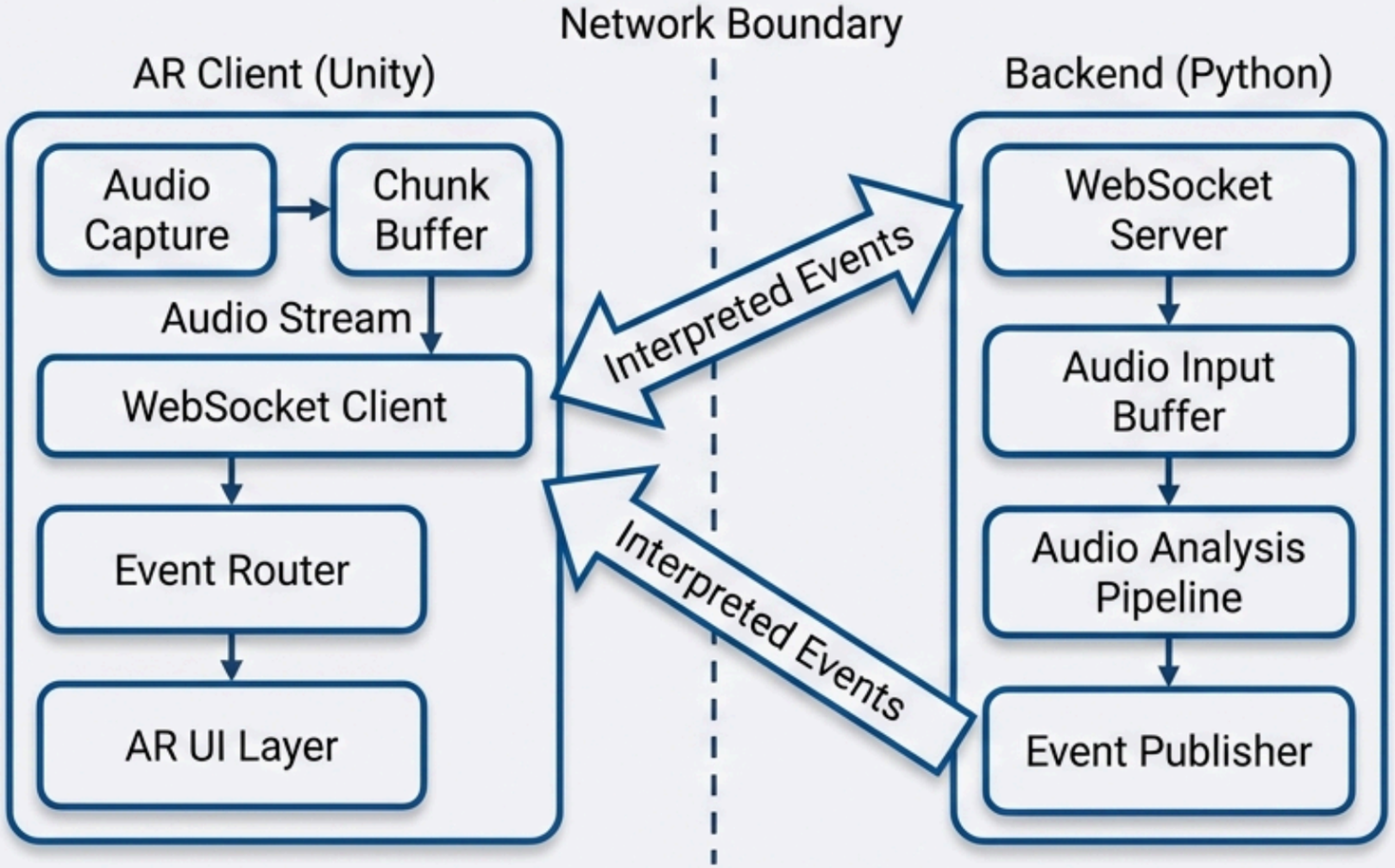
4. Visualize: Icons and text on phone screen

The Two-Part Distributed Architecture

Offloading AI processing to backend for mobile performance.

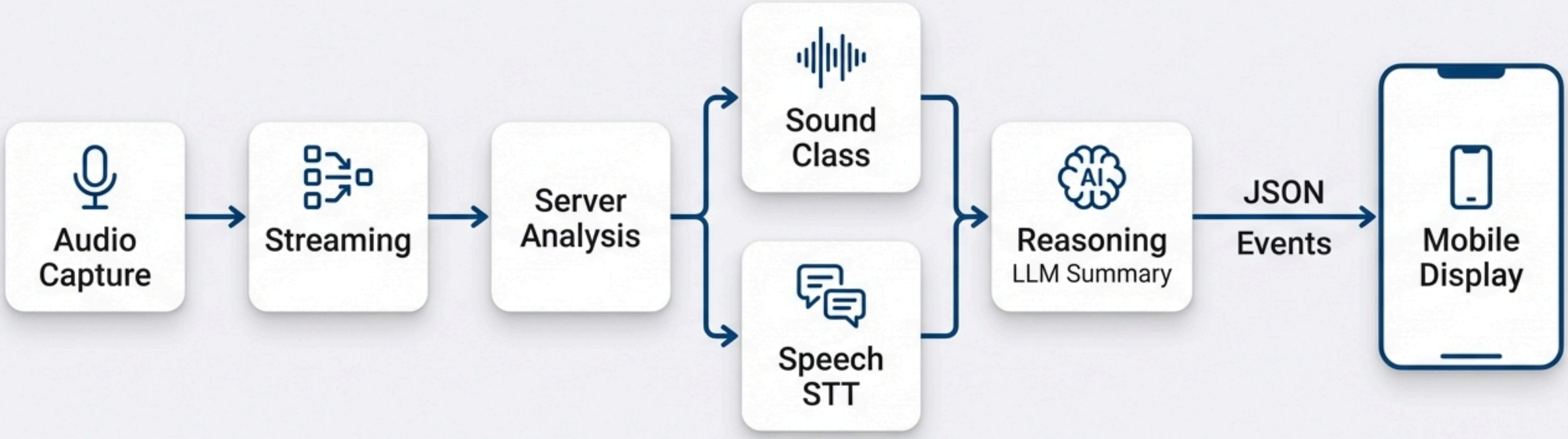
- Captures Audio (FMOD)
- Renders AR UI

- Captures Audio
- Renders AR UI
- Handles Interactions



- Heavy ML Models (YAMNet, Whisper)
- LLM Reasoning (Gemini)
- Event Publishing

End-to-End Data Pipeline



Data flows from the mobile microphone to the server for analysis, then returns as structured events.

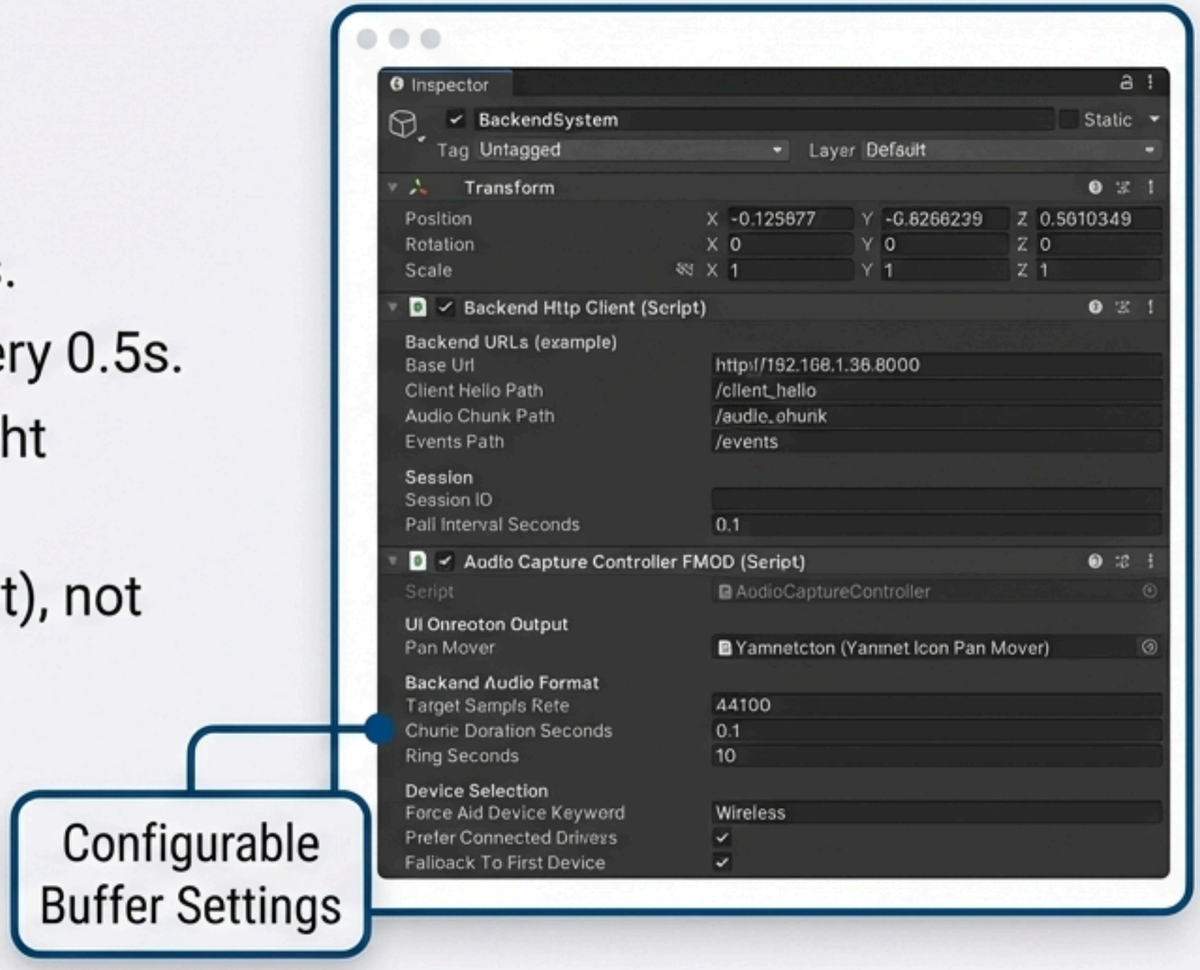
Mobile Audio Capture via FMOD

Technology: FMOD Audio Engine

Technique: Continuous Ring Buffer

- **Loop Recording:** Never stops to save files.
- **Chunking:** Extracts latest audio frame every 0.5s.
- **Stereo Estimation:** Calculates Left vs. Right energy (RMS).
- **Note:** Provides lateral direction (Left/Right), not full 360° localization.

`Target Sample Rate: 16000 , Chunk Duration: 0.1s, Ring Seconds: 10s`



Backend Analysis: Sound Classification

Model Details

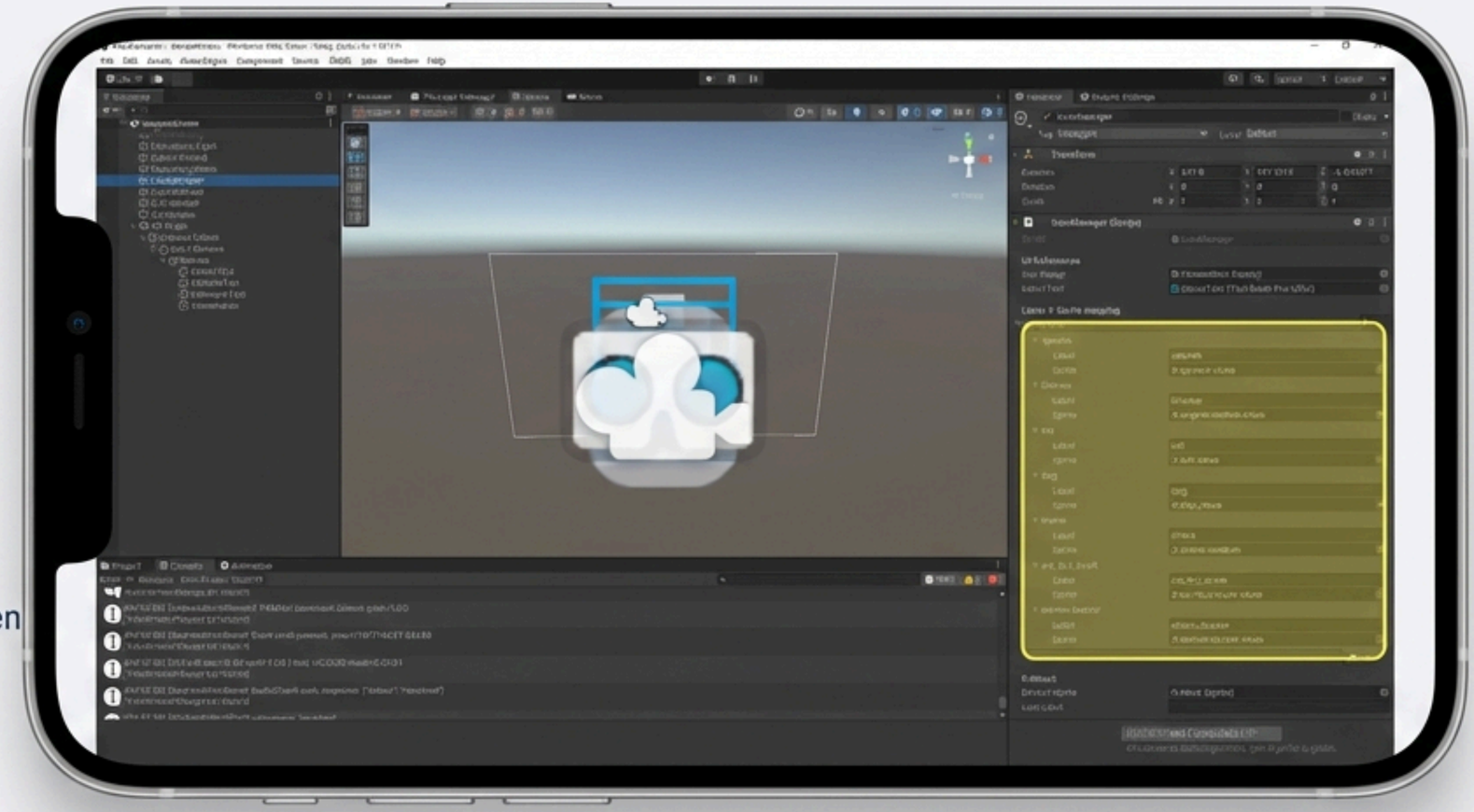
Model: YAMNet (MobileNet_v1)
Input: Raw audio chunks
Function: Classifies 521+ AudioSet categories.

Optimization: Label Mapping

Raw labels are complex. We map them to user-friendly categories.

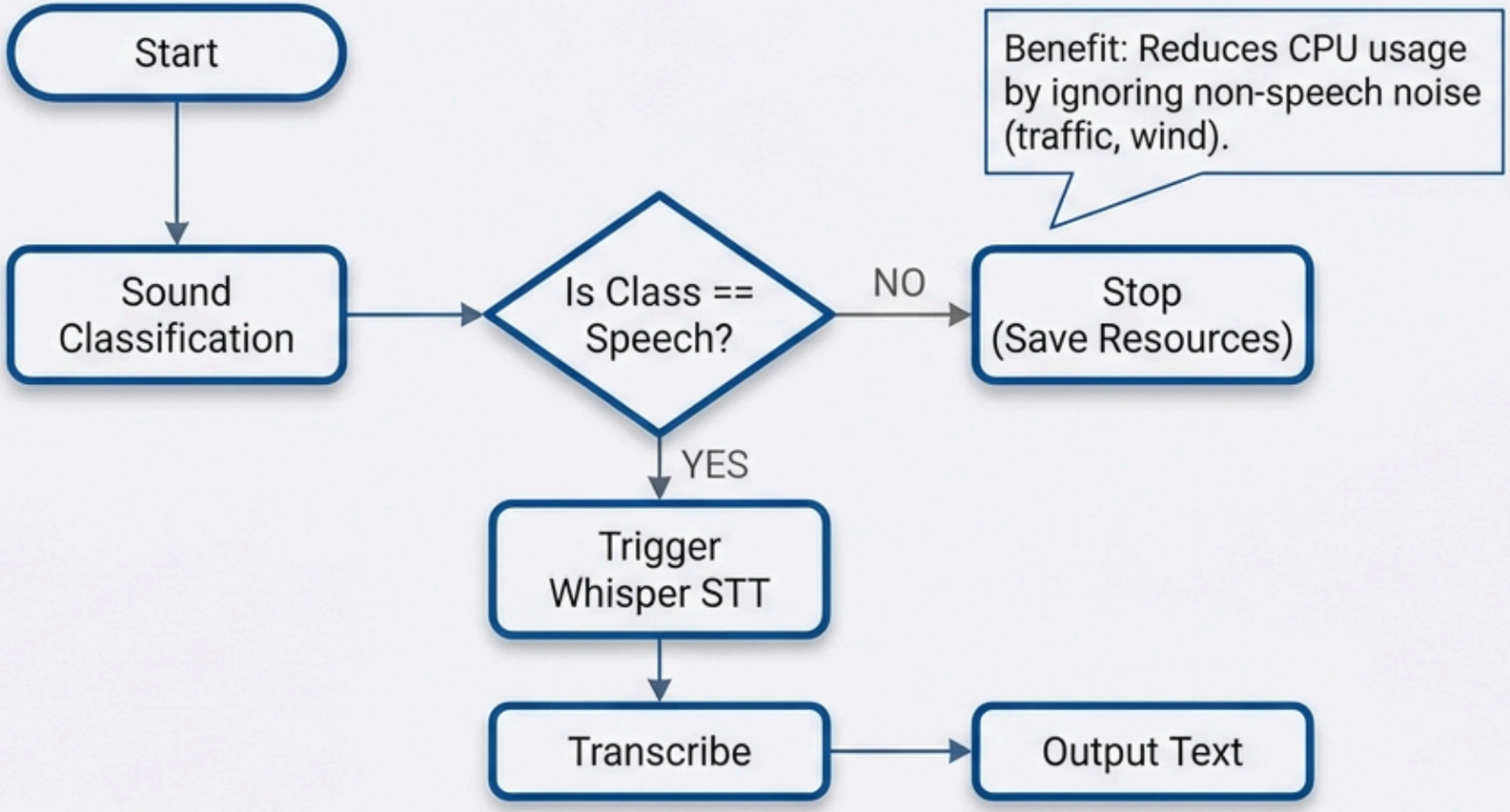
Example:

```
Raw: 'Emergency Vehicle' -> Mapped: 'Siren
Raw: 'Domestic Animal' -> Mapped: 'Dog'
```



Intelligent Speech Transcription

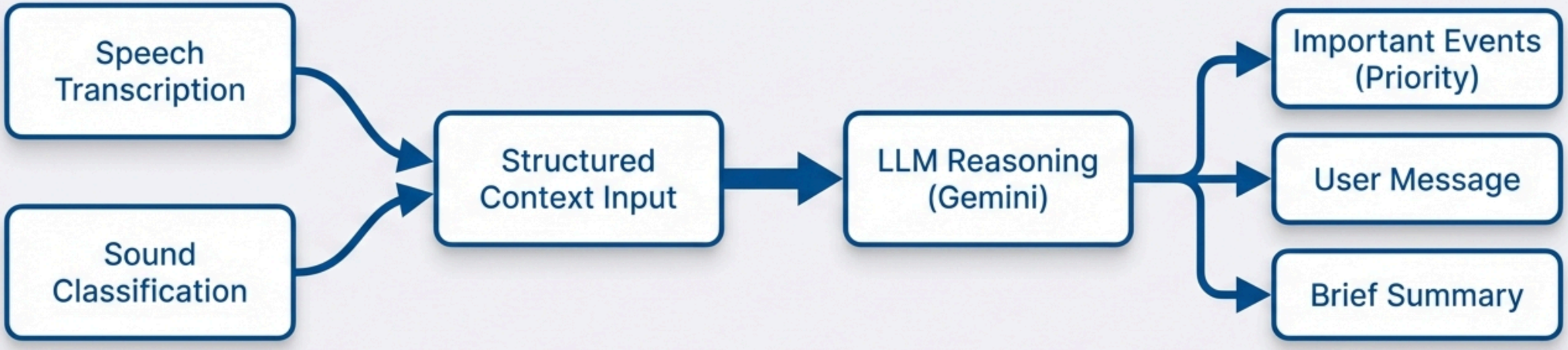
Optimizing resources with conditional logic.



Model: OpenAI Whisper (Faster-Whisper variant).

High-Level Contextual Reasoning

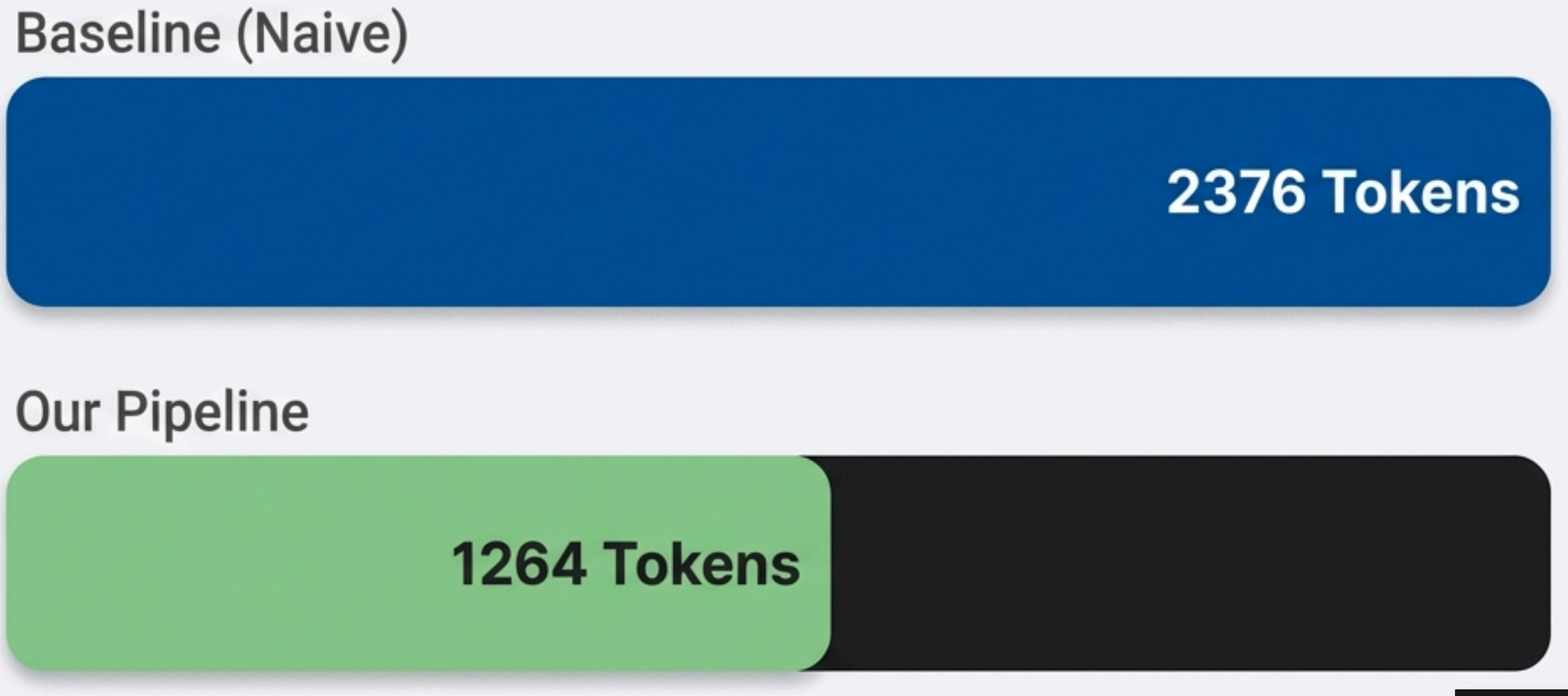
Synthesizing data into meaning.



Input Data: `'Siren Sound' (YAMNet) + Text: 'Watch out!' (Whisper)`

LLM Output: `"An emergency vehicle is approaching, and someone is warning you."`

Pipeline Efficiency & Token Optimization



46.8%
Reduction in Input Tokens

1,112
Tokens Saved
(per 74s session)

**Data based on an example session duration of 74.3 seconds.*

Efficiency achieved by filtering silence with YAMNet and processing structured summaries instead of raw audio.

Our Optimization Goal

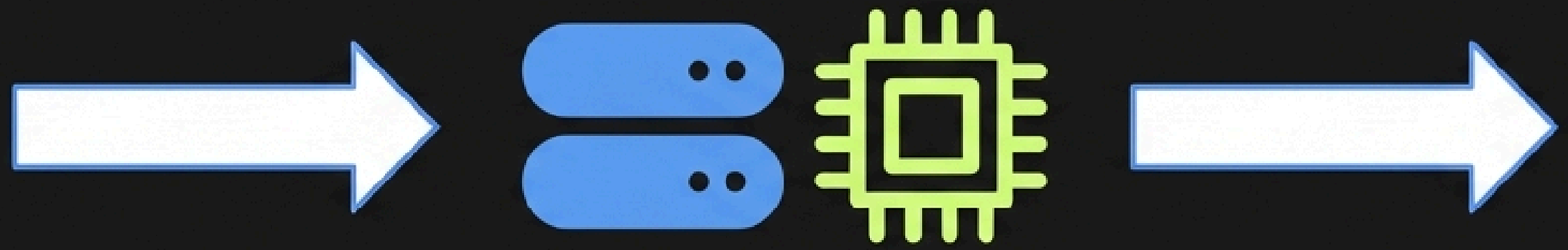
The objective is to quantify optimization benefits by comparing two distinct approaches:

1. **Our Optimized Pipeline**: The new, efficient method.
2. **The Naïve Baseline**: The standard method that sends raw audio directly to the Large Language Model (LLM).

How much efficiency do we gain by pre-processing audio data?

The Baseline (Naïve) Approach

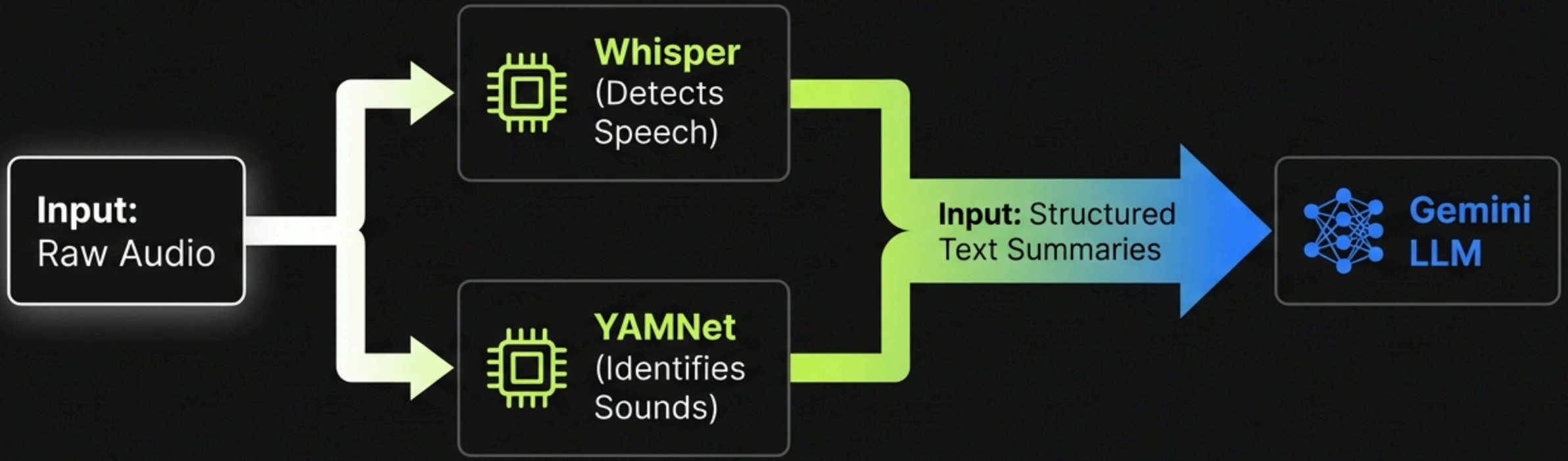
Input: Raw
Audio Stream



Roboto Mono bottleneck:
- Transcription
- Sound Understanding
- Scenario Description

Status: High computational burden (Processing raw bytes)

The Optimized Pipeline



Result: Significantly reduces the computational burden.

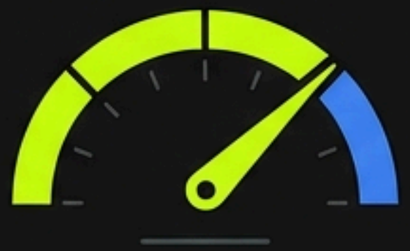
Architecture Comparison

	Baseline (Naïve)	Optimized Pipeline
Input Data	Raw Audio Data	Structured Text Summaries
Processing	Single Step (Gemini Pro 1.5)	Modular Step (Whisper + YAMNet) -> Gemini
Task Load	Transcription + Analysis + Description (Simultaneous)	Analysis only (Transcription pre-processed)

Measurement Methodology

Runtime Logging

Logs exact pipeline duration during real-time operation sessions.



Token Estimation

Estimates raw audio tokens (approx. 32 tokens per second for Gemini).

Comparative Analysis

Directly compares the estimated baseline against actual Gemini prompt tokens used.

Key Takeaways

Scalability

Lower token usage enables more concurrent operations (we can run more streams at once).

Cost Savings

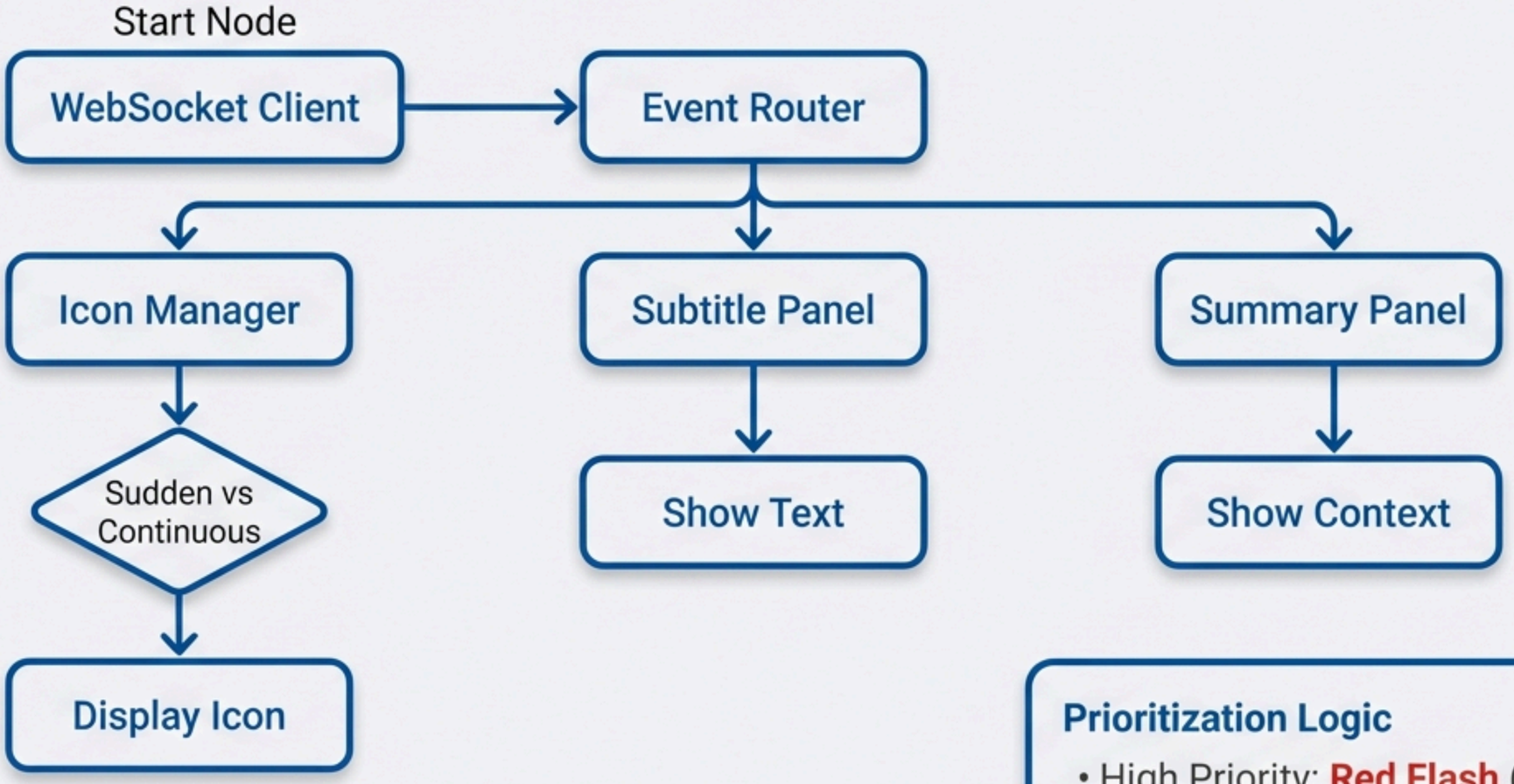
Significantly reduces API consumption costs.

Efficiency

Modular design maintains high situational awareness while using fewer resources.

Mobile Visualization: The Event Router

Deciding what to show on the AR Interface

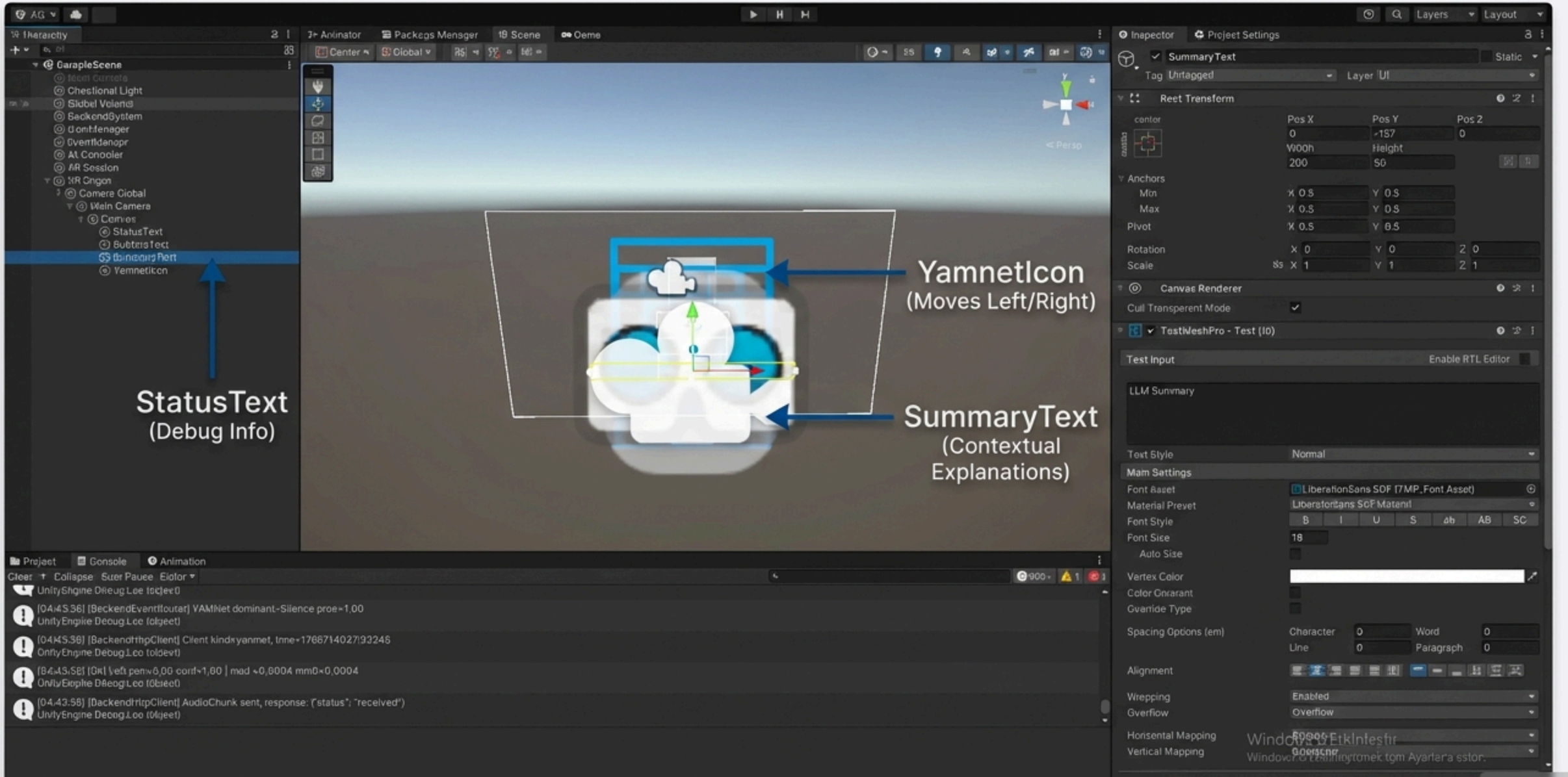


Conditional: Sudden vs Continuous

Prioritization Logic

- High Priority: **Red Flash** (e.g., Alarm)
- Low Priority: **Steady Icon** (e.g., AC Hum)

Unity UI Implementation



Unity UI Implementation

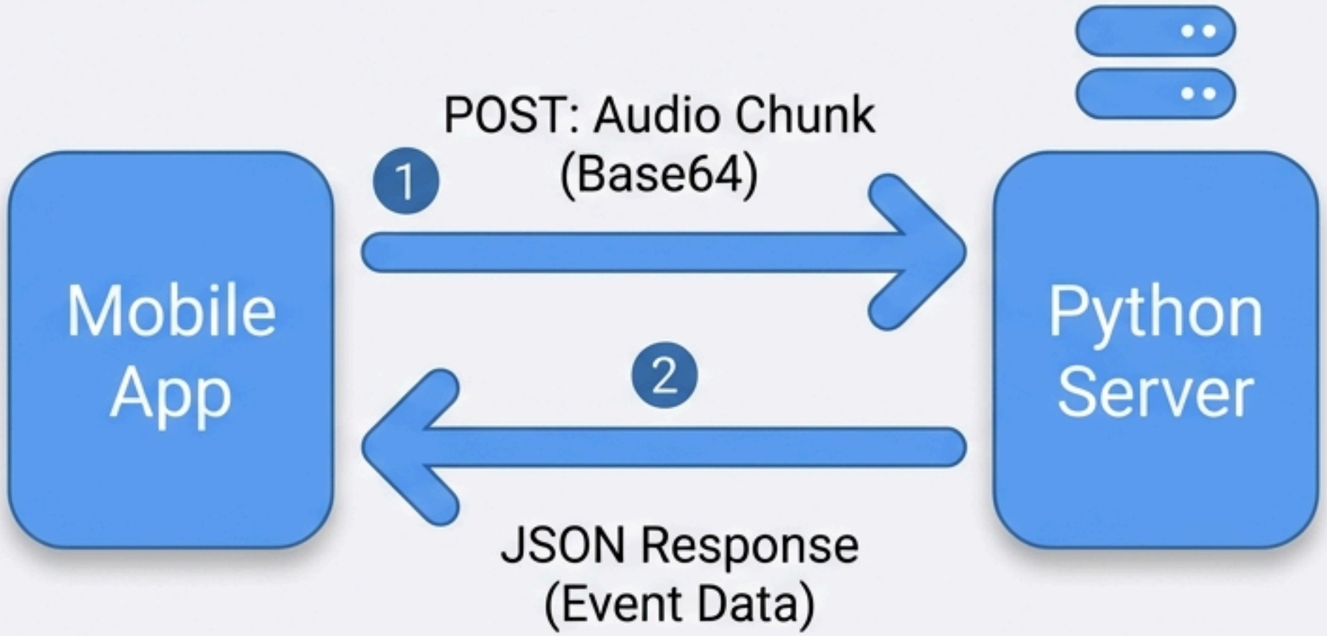
Client-Server Communication Protocol

Method: HTTP Polling (Request/Response)

Data Format: JSON

The Loop:

1. POST: audio_chunk
2. GET: new_events
3. Response: Labels, Text, Priorities




```
{
  'timestamp': 123456,
  'event_type': 'siren',
  'priority': 'high'
}
```

Technical Constraints & Realities

Latency

Sound detection is immediate (Icons). LLM summaries are asynchronous (Context). We prioritize instant icons over text.

Audio Localization

Using mobile microphone (Stereo).
Limitation: Can calculate Left/Right direction, but not Front/Back.

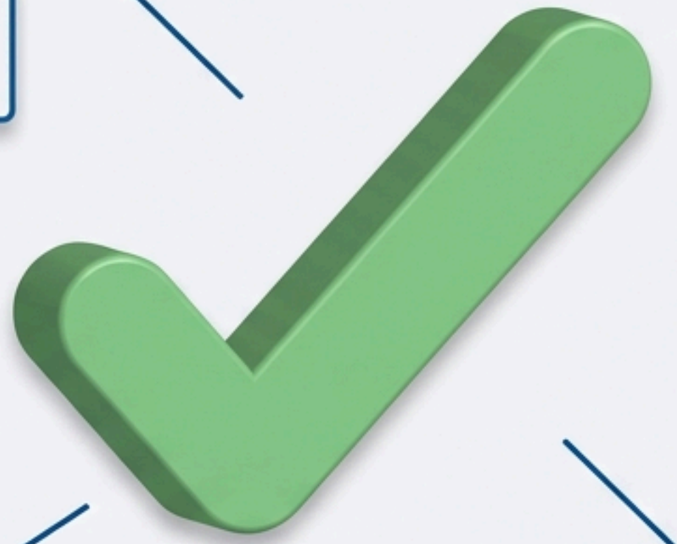
Output Device

Optimized for Android Mobile phones (Handheld AR), Meta Quest 2/3 headsets.

System Achievements

Successfully Integrated
Audio Sensing + AI + AR Visualization

Mobile-Ready
Heavy processing offloaded to backend



Efficient Pipeline
Gating & Token Reduction

User-Centric
Combined icons for safety and text for social cues

Technology Stack

Frontend (Mobile)

- Unity 2022 (AR Foundation)
- FMOD (Audio Capture)

Backend (Server)

- Python 3.x

AI Models

- Sound: YAMNet (TensorFlow)
- Speech: Faster-Whisper
- Logic: Gemini (LLM)

Communication

- HTTP/REST
- JSON Data Exchange